



HACKATHON HANDOUT

AI-Powered IT Incident Resolution Agent

Design, build, trace, containerize and deploy a production-ready LangGraph agent

Hackathon Time	10:30–17:30
Core Stack	Python • LangGraph • LLM • Langfuse • Docker • API deployment

Mission

Build an observable, controlled agentic system that can triage, investigate, diagnose, remediate and verify realistic IT incidents.

1. Business Scenario

A large enterprise operates several business-critical digital services. IT Operations receives incidents from users and monitoring systems covering application failures, authentication problems, database issues, infrastructure degradation and payment failures.

The current process is highly manual. Engineers classify incidents, inspect logs and metrics, search operational knowledge, determine likely root causes, propose remediation actions and verify recovery. The organization wants to evaluate whether an Agentic AI service can reduce triage and investigation time while preserving appropriate controls over risky actions.

2. Your Challenge

Design and implement an AI-Powered IT Incident Resolution Agent. The solution must be a stateful, multi-pattern LangGraph workflow—not a single prompt followed by an LLM response.

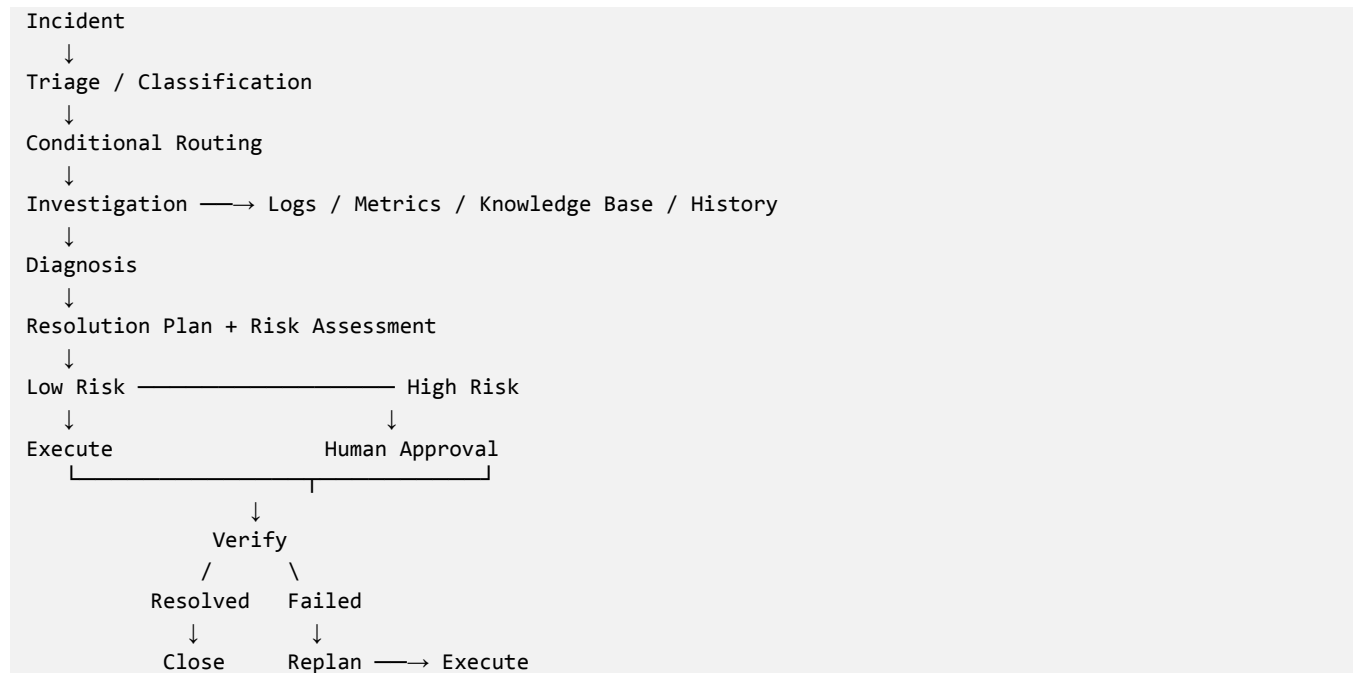
A typical input may look like:

```
Incident ID: INC-1042
Service: payment-service
Severity: Unknown
Description: Customers report payment failures for approximately 15 minutes.
Error: Database connection timeout.
```

The final output must provide, at minimum:

- Incident classification and severity
- Evidence collected during investigation
- Probable root cause
- Recommended remediation and risk level
- Approval status where required
- Execution and post-action verification result
- Final incident status and concise resolution report

3. Target Workflow



Teams may alter the architecture, but the implemented system must demonstrate the required capabilities below.

4. Mandatory Functional Requirements

ID	Requirement
FR01	Accept a structured IT incident.
FR02	Maintain workflow state using LangGraph.
FR03	Classify the incident and determine severity.
FR04	Route processing according to incident type or diagnostic evidence.
FR05	Invoke at least three meaningful tools.
FR06	Collect diagnostic evidence from simulated enterprise systems.
FR07	Generate a probable root cause supported by collected evidence.
FR08	Produce a remediation plan and assign a risk level.
FR09	Use conditional routing for decisions.
FR10	Require human approval for high-risk remediation.
FR11	Execute a simulated remediation action.
FR12	Verify whether the remediation resolved the incident.
FR13	Replan or retry when verification fails.
FR14	Produce a structured final incident report.
FR15	Handle at least one tool or workflow failure without crashing.

5. Required LangGraph Patterns

Demonstrate at least five of the following patterns:

- Sequential workflow
- Conditional routing
- Parallel or fan-out investigation
- Tool calling
- Human-in-the-loop approval
- Loop / retry / replanning
- Multi-agent or specialized-node collaboration

6. Tools and Data

Real production infrastructure is not required. Use mock data, JSON files, SQLite, in-memory structures or other controlled simulations. Implement at least three tools. Examples:

```
search_logs(service)
get_service_metrics(service)
search_knowledge_base(query)
get_incident_history(service)
restart_service(service)
check_service_health(service)
```

Remediation actions must be simulated. Do not connect the hackathon solution to real production systems.

7. Observability and Tracing

Integrate Langfuse so judges can inspect the execution path. The trace should expose the major workflow steps, LLM calls and tool executions.

- Trace / observation structure
- LLM calls and latency
- Token usage
- Tool executions
- Errors or failed paths

Optional bonus: create a Grafana view using Langfuse/ClickHouse data, such as total traces, average latency, token consumption or failures.

8. Containerization and Deployment

The completed solution must be containerized and runnable from a clean environment.

- Dockerfile
- docker-compose.yml (where required)
- .env.example with no real credentials
- README setup/run instructions

Expose an API or equivalent callable service. Recommended endpoints:

```
POST /incidents
GET  /incidents/{id}
GET  /health
```

9. Testing Expectations

Provide automated tests appropriate to the implemented solution. Minimum target:

- 3 unit tests
- 2 workflow/integration tests
- 1 end-to-end incident scenario
- At least one failure/replanning path

10. Incident Scenarios

All teams receive the same visible scenarios for development. A hidden scenario may be supplied near the end to test generalization rather than hard-coded behavior.

A — Payment Failure

```
Service: payment-service
Symptoms:
• Payment failures increased to 18%
• Database connection timeout
• Database connections at 100%

Likely diagnostic direction:
Database connection pool exhaustion
```

B — Authentication Failure

```
Service: identity-service
Symptoms:
• Login failures increased
• Token validation errors
• Identity provider latency normal

Likely diagnostic direction:
Signing/configuration problem
```

C — Performance Degradation

```
Service: order-service
Symptoms:
• Response time increased from 300 ms to 4.8 sec
• CPU utilization is normal
• Database query latency increased substantially

Likely diagnostic direction:
Database/query degradation
```

11. Proposed Hackathon Schedule — 10:30 to 17:30

Time	Session	Activity
10:30–12:00	A	Architecture & Core Agent Build — Briefing, team roles, requirements analysis, architecture, LangGraph state design, core workflow, initial tools
12:00–12:15	Break	
12:15–13:45	B	Agentic Workflow & Incident Resolution — Complete tools, conditional routing, diagnosis, remediation planning, HITL approval, verification and replanning
13:45–14:15	Lunch Break	
14:15–15:45	C	Testing, Observability & Containerization — Unit/integration testing, failure handling, Langfuse tracing, observability, API integration and Dockerization
15:45–16:00	Break	
16:00–17:30	D	Deployment, Evaluation & Demo — Deployment, end-to-end verification, hidden incident scenario, final fixes and team demonstrations

12. Suggested Team Organization

Each team has 5 participants. Teams may organize differently, but parallel ownership is strongly recommended.

Participant	Primary responsibility
1	Technical lead / architecture / integration
2	LangGraph state, routing and orchestration
3	LLM prompts, structured outputs and agent/tool behavior
4	Tools, mock systems, tests and evaluation
5	Observability, Docker, API and deployment

13. Required Deliverables

Submit one repository per team containing, as applicable:

```
README.md
architecture/
src/
tests/
Dockerfile
docker-compose.yml
.env.example
pyproject.toml
```

The final demonstration must show the complete chain:

```
Incident → LangGraph workflow → Tools → Diagnosis → Remediation decision
→ Approval (if required) → Execution → Verification / Replanning
→ Structured resolution → Langfuse trace → Running container/deployment
```

14. Assessment — 100 Points

Assessment area	Points
Business workflow correctness	10
LangGraph architecture and state design	20
Agent and tool design	15
Conditional routing and replanning	10
Human-in-the-loop and safety controls	10
Testing and robustness	10
Langfuse observability	10
Containerization	5
Deployment and end-to-end execution	10
TOTAL	100

Bonus consideration may be given for a useful Grafana dashboard, advanced evaluation, strong failure recovery or another technically meaningful enhancement. Bonus work should not compensate for missing mandatory requirements.

15. Definition of Done

A plausible LLM answer is not sufficient. The project is complete when the team demonstrates a controlled, stateful and observable operational workflow.

```

Incident
↓
Stateful LangGraph
↓
Classification / Routing
↓
Tool-assisted Investigation
↓
Evidence-based Diagnosis
↓
Remediation + Risk Decision
↓
HITL where required
↓
Simulated Execution
↓
Verification
↓
Replanning when necessary
↓
Structured Resolution
↓
Langfuse Trace
↓
Containerized and Deployed Service

```

16. Ground Rules

- Prioritize a working end-to-end solution over excessive feature breadth.
- Do not hard-code the visible incident answers into the workflow.
- Do not expose API keys, passwords or other secrets in source control.
- All remediation actions must be simulated or executed only in the supplied sandbox.
- Every team member should be able to explain the architecture and their contribution.
- Judging focuses on engineering quality, workflow behavior, robustness and demonstrability—not UI polish.

Build an agentic operational system—not an incident chatbot.